

Intro_Numba_CUDA

March 6, 2026

1 Programación de GPUs en Python con Numba CUDA

2 Confirmar que disponemos de GPU

```
[1]: #!/usr/bin/env python3
# Install numba-cuda==0.4.0
# from numba import config
# config.CUDA_ENABLE_PYNVJITLINK = 1
```

```
[2]: !nvidia-smi
```

Wed Feb 25 06:12:55 2026

```
+-----+
+-----+
| NVIDIA-SMI 580.82.07                Driver Version: 580.82.07          CUDA Version:
13.0 |
+-----+-----+-----+
+-----+
| GPU Name                               Persistence-M | Bus-Id        Disp.A | Volatile
Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap |      Memory-Usage | GPU-Util
Compute M. |
|                               |                      |
MIG M. |
|=====+-----+-----+=====
=====|
|  0  Tesla T4                               Off |  00000000:00:04.0 Off |
0 |
| N/A   64C    P8               12W /  70W |      0MiB /  15360MiB |      0%
Default |
|                               |                      |
N/A |
+-----+-----+-----+
+-----+
+-----+
| Processes:
|
```

GPU	GI	CI	PID	Type	Process name
GPU Memory					
	ID	ID			
Usage					

```

=====
=====|
| No running processes found
|
+-----+
-----+

```

Nota: En Google Colab si no aparece información de la GPU, debemos ir a Menu -> Entorno de Ejecución -> Cambiar tipo de entorno de ejecución -> Seleccionar GPU T4 -> Guardar

3 Dispositivos CUDA

Numba admite GPU habilitadas para CUDA con capacidad de cómputo 2.0 o superior con un controlador Nvidia actualizado. Necesitará tener instalada la versión 8.0 o posterior del kit de herramientas CUDA.

La extensión cuda admite casi todas las funciones de CUDA, con excepción del paralelismo dinámico y la memoria de texturas. El paralelismo dinámico permite ejecutar un núcleo de cómputo desde otros núcleos de cómputo. La memoria de texturas tiene un patrón de almacenamiento en caché basado en la localidad espacial. No entraremos en detalles aquí.

Primero verifiquemos qué tipo de dispositivo Cuda tenemos en el sistema.

```
[3]: from numba import cuda
     cuda.detect()
```

```

Found 1 CUDA devices
id 0          b'Tesla T4'          [SUPPORTED]
        Compute Capability: 7.5
        PCI Device ID: 4
        PCI Bus ID: 0
        UUID:
GPU-0cb26f1c-ba0b-770e-5f90-d8a4fba66d3e
        Watchdog: Disabled
        FP32/FP64 Performance Ratio: 32
Summary:
    1/1 devices are supported

```

```
[3]: True
```

3.1 Lanzamiento de kernels

Lanzar un kernel Cuda desde Numba es muy fácil. Un kernel se define utilizando el decorador @cuda.jit

```
[4]: @cuda.jit
def an_empty_kernel():
    """A kernel that doesn't do anything."""
    # Get my current position in the global grid
    [pos_x, pos_y] = cuda.grid(2)
```

```
[5]: an_empty_kernel
```

```
[5]: CUDADispatcher(<function an_empty_kernel at 0x7eabb8ec4680>)
```

Para poder iniciar el kernel, necesitamos especificar la disposición de los threads. Los siguientes comandos definen una disposición de threads en dos dimensiones de 16x16 threads por bloque y 256x256 bloques. En total esto nos da 16.777.216 threads. Esto parece mucho, pero las GPU están diseñadas para ejecutar grandes cantidades de threads. La única restricción es que se nos permite tener como máximo 1024 threads por bloque.

```
[ ]: threadsperblock = (16, 16) # Should be a multiple of 32 if possible.
      blockspergrid = (256, 256) # Blocks per grid
```

Ahora podemos iniciar los 16,8 millones de threads llamando

```
[7]: an_empty_kernel(blockspergrid, threadsperblock)()
```

Dentro de un kernel podemos utilizar los siguientes comandos para obtener la posición del hilo.

```
[8]: @cuda.jit
def another_kernel():
    """Commands to get thread positions"""
    # Get the thread position in a thread block
    tx = cuda.threadIdx.x
    ty = cuda.threadIdx.y
    tz = cuda.threadIdx.z

    # Get the id of the thread block
    block_x = cuda.blockIdx.x
    block_y = cuda.blockIdx.y
    block_z = cuda.blockIdx.z

    # Number of threads per block
    dim_x = cuda.blockDim.x
    dim_y = cuda.blockDim.y
    dim_z = cuda.blockDim.z

    # Global thread position
    pos_x = tx + block_x * dim_x
    pos_y = ty + block_y * dim_y
    pos_z = tz + block_z * dim_z

    # We can also use the grid function to get
```

```

# the global position

(pos_x, pos_y, pos_z) = cuda.grid(3)
# For a 1-or 2-d grid use grid(1) or grid(2)
# to return a scalar or a two tuple.

threadsperblock = (16, 16, 4) # Should be a multiple of 32 if possible.
blockspergrid = (256, 256, 256) # Blocks per grid

another_kernel[blockspergrid, threadsperblock]()

```

Numba admite en los kernels solo un conjunto seleccionado de funciones que son compatibles con el estándar CUDA. No se permiten excepciones, administradores de contexto, listas por comprensión ni declaraciones yield. Los tipos admitidos son int, float, complex, bool, None, tuple. Para obtener una descripción general completa de las funciones admitidas, consulte <https://numba.pydata.org/numba-doc/dev/cuda/cudapysupported.html#> . Solo se admite un pequeño conjunto de funciones Numpy. Básicamente, todo lo que requiera administración de memoria dinámica no funcionará debido a las restricciones de los kernels del modelo de programación Cuda.

3.2 Gestión de memoria

Para los núcleos simples, podemos confiar en que Numba copie datos hacia y desde el dispositivo. Para códigos más complejos, necesitamos administrar manualmente los búferes en el dispositivo.

- Copiar datos al dispositivo

```

[9]: import numpy as np

arr = np.arange(10)
device_arr = cuda.to_device(arr)

```

- Copiar datos del dispositivo al host

```

[10]: host_arr = device_arr.copy_to_host()

```

- Copiar en una matriz existente

```

[11]: host_array = np.empty(shape=device_arr.shape, dtype=device_arr.dtype)
device_arr.copy_to_host(host_array)

```

```

[11]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

```

- Generar una nueva matriz en el dispositivo

```

[12]: device_array = cuda.device_array((10,), dtype=np.float32)

```

3.3 Funciones avanzadas

CUDA cuenta con una serie de funciones avanzadas compatibles con Numba. Algunas de ellas son:

- La memoria pinned es una forma de asignación de memoria que permite una transferencia de datos mucho más rápida que los buffers estándar.
- Los streams son una forma de ejecutar múltiples tareas en una GPU de manera concurrente. De manera predeterminada, CUDA ejecuta un comando tras otro en el dispositivo. Los streams nos permiten crear varias colas concurrentes para programar tareas en el dispositivo. Esto permite, por ejemplo, tener un stream de kernel que realiza cálculos y un stream de memoria que realiza transferencias de memoria, de manera concurrente. Se pueden usar eventos para sincronizar entre diferentes streams.
- Numba admite varios dispositivos. Existen rutinas auxiliares para enumerar y seleccionar diferentes dispositivos.

Para obtener una lista completa de características, consulte la guía en <https://numba.readthedocs.io/en/stable/cuda/index.html>